



دانشگاه صنعتی امیرکبیر

(پلی تکنیک تهران)

دانشکده ریاضی و علوم کامپیوتر

پایان نامه کارشناسی ارشد

الگوریتم دایکسترا

نگارش

رضا میراحمدی

استاد راهنما

استاد حسینقلی زاده

خرداد ۱۴۰۵

بِسْمِ اللَّهِ الرَّحْمَنِ الرَّحِيمِ

فهرست مطالب

صفحه

عنوان

۴	۲ مفاهیم مقدماتی نظریه گراف
۴	۱-۲ تعریف کراف
۵	۲-۲ مسیر
۶	۱-۲-۲ کوتاه ترین مسیر
۷	۳-۲ ماتریس مجاورت
۷	۱-۳-۲ در گراف بدون وزن
۷	۲-۳-۲ در گراف وزن دار
۹	۳ الگوریتم دایکسترا
۹	۱-۳ مقدمه
۹	۲-۳ صورت مسئله
۱۰	۳-۳ ورودی و خروجی الگوریتم
۱۰	۴-۳ ایده اصلی الگوریتم
۱۱	۵-۳ مراحل اجرا، شبه کد و فلوچارت الگوریتم دایکسترا
۱۴	۴ تحلیل ریاضی الگوریتم دایکسترا
۱۵	۱-۴ درستی الگوریتم
۱۵	۲-۴ پیچیدگی زمانی و حافظه ای
۱۵	۳-۴ مزایا و محدودیت ها
۱۷	۵ جمع بندی و نتیجه گیری و پیشنهادات
۱۷	۱-۵ پیشنهادات
۱۸	منابع و مراجع

شکل	فهرست اشکال	صفحه
۱-۲	گراف وزن دار	۵
۱-۳	فلوچارت الگوریتم دیکسترا	۱۳

صفحه	فهرست جداول	جدول
۱۱		۱-۳ نمونه اجرای الگوریتم دایکسترا

فصل ۱

مقدمات الگوریتم دایکسترا

۱-۱ مقدمه

در بسیاری از مسائل دنیای واقعی، یافتن کوتاه‌ترین مسیر بین دو نقطه یکی از مهم‌ترین چالش‌ها محسوب می‌شود. این مسئله در قالب گراف‌ها مدل‌سازی می‌شود؛ به‌طوری‌که نقاط به‌صورت راس و مسیرهای بین آن‌ها به‌صورت یال نمایش داده می‌شوند. هر یال می‌تواند دارای وزنی باشد که نشان‌دهنده هزینه، فاصله، زمان یا هر معیار دیگری باشد. هدف مسئله کوتاه‌ترین مسیر، یافتن مسیری بین یک راس مبدا و سایر راس‌ها (یا یک راس مقصد مشخص) است به‌گونه‌ای که مجموع وزن یال‌های طی‌شده حداقل باشد.

۲-۱ اهمیت یافتن کوتاه‌ترین مسیر

مسئله کوتاه‌ترین مسیر یکی از بنیادی‌ترین مسائل در نظریه گراف و علوم کامپیوتر است. اهمیت این مسئله به دلیل کاربرد گسترده آن در حوزه‌های مختلف علمی، صنعتی و مهندسی می‌باشد. یافتن مسیر بهینه باعث کاهش موارد زیر می‌شود:

- زمان
- هزینه
- مصرف انرژی
- مصرف منابع محاسباتی

۳-۱ کاربرد های عملی

۱-۳-۱ مسیریابی

سیستم های مسیریابی برای یافتن سریع ترین مسیر بین دو نقطه از الگوریتم های کوتاه ترین مسیر استفاده می کنند. در این سیستم ها وزن یال ها می تواند نشان دهنده موارد زیر باشد:

- فاصله
- میزان ترافیک
- هزینه عوارضی
- زمان سفر

۲-۳-۱ شبکه های کامپیوتری

در شبکه های کامپیوتری، الگوریتم های مسیریابی از الگوریتم دیکسترا برای تعیین بهترین مسیر ارسال داده استفاده می کنند. در این مسئله وزن یال ها می تواند بیانگر موارد زیر باشد:

- تاخیر
- پهنای باند
- میزان ازدحام
- هزینه انتقال اطلاعات

۳-۳-۱ سیستم های حمل و نقل

در سیستم های ریلی، هوایی و شهری، برای برنامه ریزی حرکت وسایل نقلیه و بهینه سازی مسیرها از الگوریتم های کوتاه ترین مسیر استفاده می شود تا هزینه مصرف سوخت و زمان کاهش و بهره وری افزایش یابد.

۴-۱ معرفی الگوریتم دایکسترا

الگوریتم دایکسترا^۱ یکی از معروف ترین و پرکاربردترین الگوریتم ها برای حل مسئله کوتاه ترین مسیر در گراف های وزن دار با وزن های غیرمنفی است. این الگوریتم در سال ۱۹۵۶ توسط دانشمند هلندی ادوارد

¹Dijkstra's Algorithm

دیکسترا^۲ ارائه شد.

ایده اصلی الگوریتم بسیار ساده اما قدرتمند است، در هر مرحله، نزدیک‌ترین راس بررسی‌نشده به مبدا انتخاب شده و فاصله همسایگان آن به‌روزرسانی می‌شود.

²Edsger W. Dijkstra

فصل ۲

مفاهیم مقدماتی نظریه گراف

۱-۲ تعریف کراف

گراف یک ساختار ریاضی برای نمایش مجموعه‌ای از اشیا و روابط بین آن‌ها است. به‌طور کلی، گراف از دو بخش اصلی تشکیل شده است:

- راس‌ها: نمایش‌دهنده نقاط یا گره‌ها است

- یال‌ها: نمایش‌دهنده ارتباط بین راس‌ها است

به‌صورت ریاضی، اگر گراف را به‌صورت زیر در نظر بگیریم:

$$G = (V, E)$$

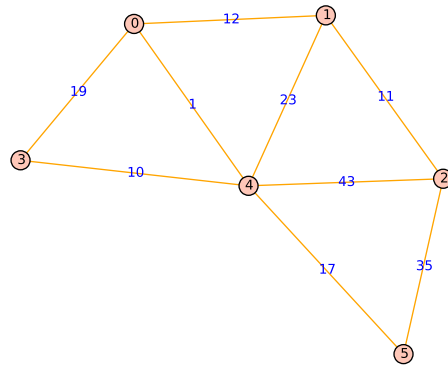
که در آن:

- V مجموعه راس‌ها

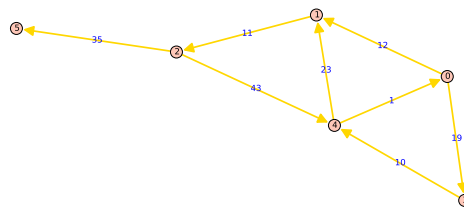
- E مجموعه یال‌ها

تابع وزن بر اساس فرضیات الگوریتم دایکسترا به‌صورت زیر تعریف شود:

$$w : E \rightarrow \mathbb{R}^+$$



(آ) مثال گراف ساده وزن دار



(ب) مثال گراف جهت دار وزن دار

شکل ۲-۱: گراف وزن دار

هدف، یافتن مسیری از راس مبدا s به راس مقصد t است که مجموع وزن‌ها کمینه باشد:

$$\min \sum_{(u,v) \in path} w(u,v)$$

۲-۲ مسیر

مسیر به دنباله‌ای از راس‌های غیر تکراری گفته می‌شود که هر دو راس متوالی در آن توسط یک یال به یکدیگر متصل باشند.

به عنوان مثال، در گرافی با راس‌های A, B, C, D ، دنباله زیر یک مسیر محسوب می‌شود:

$$A \rightarrow B \rightarrow D$$

زیرا بین A و B و همچنین بین B و D یال وجود دارد.
در گراف‌های وزن‌دار، طول یا هزینه یک مسیر برابر است با مجموع وزن یال‌های آن مسیر. به صورت کلی، اگر مسیر از راس‌های زیر تشکیل شده باشد:

$$v_1, v_2, v_3, \dots, v_k$$

هزینه مسیر برابر است با:

$$w(v_1, v_2) + w(v_2, v_3) + \dots + w(v_{k-1}, v_k)$$

۱-۲-۲ کوتاه‌ترین مسیر

کوتاه‌ترین مسیر مسیری است که از میان تمام مسیرهای ممکن بین دو راس، کمترین مجموع وزن را داشته باشد. فرض کنید بخواهیم از راس A به راس D برسیم. اگر دو مسیر زیر وجود داشته باشد:

$$1. A \rightarrow B \rightarrow D \text{ با هزینه } 7$$

$$2. A \rightarrow C \rightarrow D \text{ با هزینه } 5$$

در این صورت مسیر دوم کوتاه‌ترین مسیر است، زیرا مجموع وزن‌های آن کمتر است. مسئله کوتاه‌ترین مسیر ممکن است به شکل‌های مختلف مطرح شود، از جمله:

- کوتاه‌ترین مسیر از یک راس مبدا به یک راس مقصد
- کوتاه‌ترین مسیر از یک راس مبدا به همه راس‌ها
- کوتاه‌ترین مسیر بین همه جفت راس‌ها

الگوریتم دیکسترا برای یافتن کوتاه‌ترین مسیر از یک راس مبدا به سایر راس‌ها طراحی شده است.

۳-۲ ماتریس مجاورت

یکی از روش‌های نمایش گراف، استفاده از **ماتریس مجاورت** است. اگر گرافی دارای n راس باشد، ماتریس مجاورت آن یک ماتریس $n \times n$ است که درایه سطر i و ستون j نشان می‌دهد آیا بین راس i و راس j یال وجود دارد یا خیر.

۱-۳-۲ در گراف بدون وزن

- اگر بین دو راس یال وجود داشته باشد، مقدار درایه برابر ۱ قرار می‌گیرد.
- در غیر این صورت مقدار آن ۰ است.

۲-۳-۲ در گراف وزن دار

- اگر بین دو راس یال وجود داشته باشد، مقدار درایه برابر وزن یال قرار می‌گیرد.
- اگر یالی وجود نداشته باشد، معمولاً مقدار صفر، بی‌نهایت، یا مقدار خاصی برای آن در نظر گرفته می‌شود.

به‌عنوان نمونه، فرض کنید گرافی با سه راس A ، B و C داشته باشیم که وزن یال‌ها به این صورت است:

• بین A و B : وزن ۲

• بین A و C : وزن ۵

• بین B و C : وزن ۱

ماتریس مجاورت آن به شکل زیر خواهد بود:

$$\begin{bmatrix} 0 & 2 & 5 \\ 2 & 0 & 1 \\ 5 & 1 & 0 \end{bmatrix}$$

در این ماتریس:

- سطرها و ستون‌ها نمایانگر راس‌ها هستند.
 - مقدار هر خانه نشان‌دهنده وزن یال بین دو راس است.
- ماتریس مجاورت یکی از ساده‌ترین روش‌های نمایش گراف است، اما در گراف‌های بزرگ و خلوت ممکن است باعث مصرف زیاد حافظه شود. در چنین شرایطی، استفاده از **لیست مجاورت** مناسب‌تر است.

فصل ۳

الگوریتم دایکسترا

۱-۳ مقدمه

در این فصل، الگوریتم دایکسترا به عنوان یکی از مهم‌ترین الگوریتم‌های یافتن کوتاه‌ترین مسیر در گراف‌های وزن‌دار معرفی و بررسی می‌شود. این الگوریتم برای یافتن کمینه فاصله از یک رأس مبدأ به سایر رأس‌های گراف به کار می‌رود، به شرط آن که وزن تمام یال‌ها نامنفی باشد. در ادامه، ابتدا صورت مسئله بیان می‌شود، سپس مراحل الگوریتم، شبه‌کد، روابط ریاضی و فلوچارت مربوط به آن ارائه خواهد شد.

۲-۳ صورت مسئله

اگر یک مسیر از رأس s به رأس v به صورت $P = (v_0, v_1, \dots, v_k)$ تعریف شود، به‌طوری که $v_0 = s$ و $v_k = v$ ، طول این مسیر برابر مجموع وزن یال‌های آن خواهد بود. بنابراین، طول مسیر مطابق رابطه (۱-۳) تعریف می‌شود.

$$d(P) = \sum_{i=1}^k w(v_{i-1}, v_i) \quad (1-3)$$

هدف، یافتن مسیری است که مقدار رابطه (۳-۱) را کمینه کند. اگر کوتاه‌ترین فاصله از مبدأ s تا رأس v را با $d(v)$ نمایش دهیم، آنگاه الگوریتم دایکسترا این مقادیر را برای تمام رأس‌ها محاسبه می‌کند.

۳-۳ ورودی و خروجی الگوریتم

ورودی الگوریتم شامل موارد زیر است:

- یک گراف وزن‌دار نامنفی $G = (V, E)$

- تابع وزن یال‌ها w

- و یک رأس مبدأ $s \in V$

خروجی الگوریتم شامل دو بخش است:

- کمینه فاصله از مبدأ به هر رأس

- مسیر متناظر با فاصله کمینه بین رأس مبدأ و مقصد

۴-۳ ایده اصلی الگوریتم

ایده اصلی الگوریتم دایکسترا آن است که در ابتدا فاصله مبدأ تا خودش برابر صفر و فاصله آن تا سایر رأس‌ها بی‌نهایت در نظر گرفته می‌شود. سپس در هر گام، رأسی انتخاب می‌شود که کمترین فاصله موقت را دارد. پس از انتخاب این رأس، یال‌های خروجی آن بررسی می‌شوند و اگر مسیر جدیدی کوتاه‌تر از مقدار فعلی باشد، فاصله رأس مقصد اصلاح می‌شود. فرآیند به‌روزرسانی فاصله‌ها بر اساس رابطه (۳-۲) انجام می‌شود:

$$d(v) = \min(d(v), d(u) + w(u, v)) \quad (۳-۲)$$

رابطه (۲-۳) همان عمل شل سازی^۱ را نشان می دهد که هسته اصلی الگوریتم دایکسترا است. در جدول زیر بر اساس ایده الگوریتم دایکسترا یک مثالی آورده ایم:

رأس	فاصله	والد	وضعیت
s	۰		انتخاب شد
a	۴	s	در صف
b	۲	s	در صف
c	۵	b	در صف
d	۷	c	در صف

جدول ۳-۱: نمونه اجرای الگوریتم دایکسترا

۳-۵ مراحل اجرا، شبه کد و فلوچارت الگوریتم دایکسترا

مراحل اجرای الگوریتم را می توان به صورت زیر خلاصه کرد:

۱. برای تمام رأس ها مقدار فاصله اولیه برابر بی نهایت قرار داده می شود.

۲. فاصله رأس مبدأ برابر صفر قرار داده می شود.

۳. تا زمانی که هنوز رأس بررسی نشده وجود دارد:

(آ) رأسی با کمترین فاصله موقت انتخاب می شود.

(ب) این رأس به عنوان رأس نهایی علامت گذاری می شود.

(ج) همه همسایه های آن بررسی می شوند.

(د) در صورت کوتاه تر بودن مسیر جدید، فاصله همسایه به روزرسانی می شود.

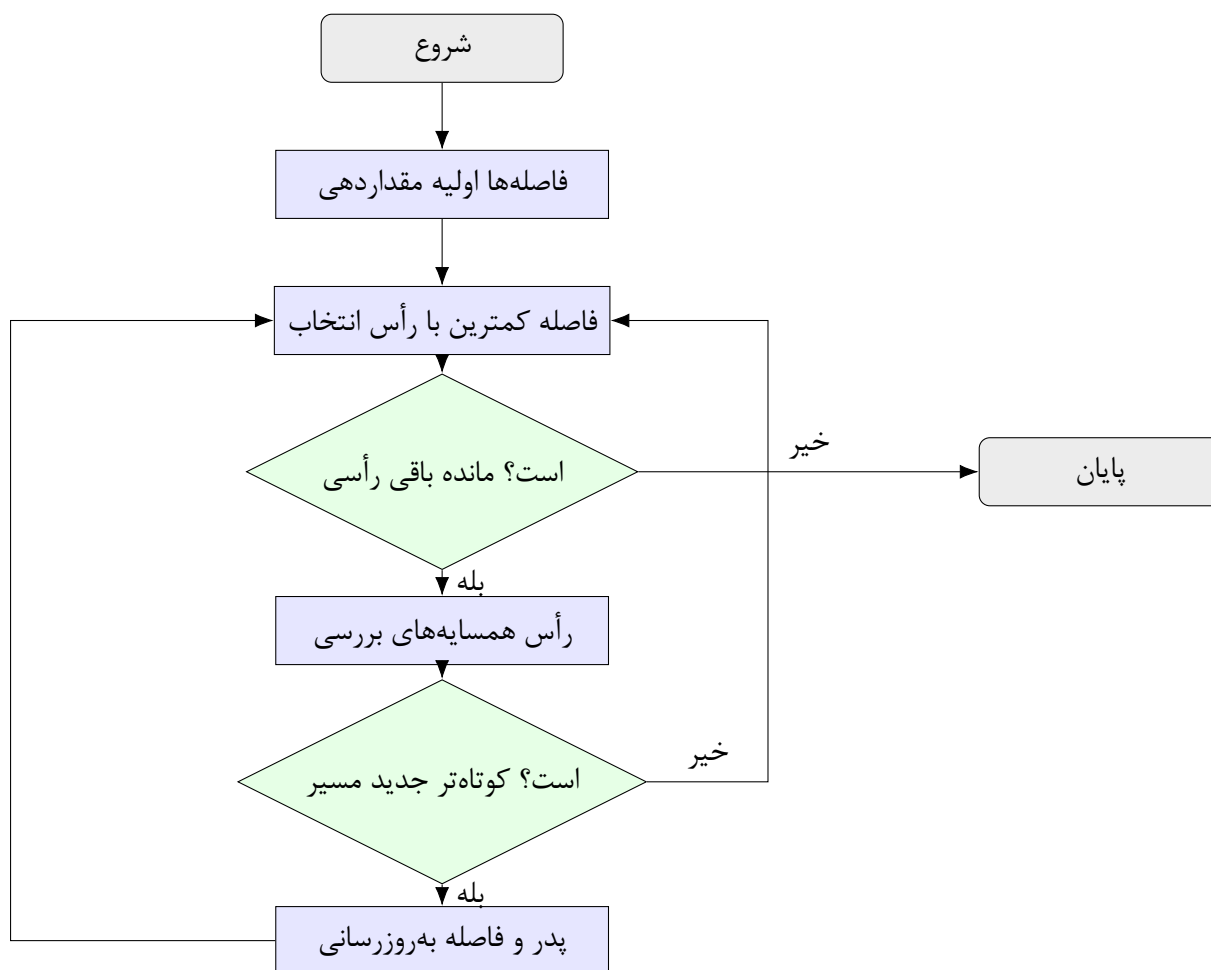
شبه کد الگوریتم دایکسترا در الگوریتم ۱ آمده است. همان طور که مشاهده می شود، در این الگوریتم ابتدا فاصله ها مقادیردهی اولیه شده و سپس با انتخاب رأس دارای کمترین فاصله، مقادیر جدید محاسبه می شوند.

¹Relaxation

الگوریتم ۱ Dijkstra's Algorithm for Shortest Paths

ورودی: $G = (V, E)$, weight function w , source vertex s
خروجی: Shortest distance from s to all vertices

```
1: for all  $v \in V$  do
2:    $dist[v] \leftarrow \infty$ 
3:    $parent[v] \leftarrow \text{null}$ 
4: end for
5:  $dist[s] \leftarrow 0$ 
6:  $Q \leftarrow V$  {set of unvisited vertices}
7: while  $Q \neq \emptyset$  do
8:    $u \leftarrow$  vertex in  $Q$  with minimum  $dist[u]$ 
9:   remove  $u$  from  $Q$ 
10:  for all neighbor  $v$  of  $u$  do
11:    if  $dist[u] + w(u, v) < dist[v]$  then
12:       $dist[v] \leftarrow dist[u] + w(u, v)$ 
13:       $parent[v] \leftarrow u$ 
14:    end if
15:  end for
16: end while
17: return  $dist$ 
```



شکل ۳-۱: فلوچارت الگوریتم دیکسترا

فصل ۴

تحلیل ریاضی الگوریتم دایکسترا

در هر مرحله از اجرای الگوریتم، فاصله یک رأس به صورت نهایی تثبیت می‌شود. اگر رأس انتخاب شده در یک گام را u بنامیم، در آن صورت مقدار $d(u)$ در همان مرحله کمینه واقعی فاصله از مبدأ تا رأس u خواهد بود. در نتیجه، پس از پایان الگوریتم، برای هر رأس v خواهیم داشت:

$$d(v) = \delta(s, v) \quad (۱-۴)$$

که در آن $\delta(s, v)$ نشان‌دهنده کوتاه‌ترین فاصله واقعی از رأس s به رأس v است. همچنین اگر برای رأس v چند مسیر جایگزین وجود داشته باشد، کمینه فاصله آن را می‌توان مطابق رابطه (۲-۴) در قالبی چندخطی نمایش داد:

$$\begin{aligned} d(v) = \min \big(& d(u_1) + w(u_1, v), \\ & d(u_2) + w(u_2, v), \\ & d(u_3) + w(u_3, v), \\ & \dots, \\ & d(u_m) + w(u_m, v) \big) \end{aligned} \quad (۲-۴)$$

۱-۴ درستی الگوریتم

درستی الگوریتم دیکسترا مبتنی بر این اصل است که وقتی یک رأس با کمترین فاصله موقت انتخاب می‌شود، دیگر هیچ مسیر کوتاه‌تری به آن رأس از طریق رأس‌های بررسی نشده وجود نخواهد داشت. این ویژگی تنها زمانی برقرار است که وزن تمام یال‌ها نامنفی باشند. بنابراین، یکی از پیش‌فرض‌های اساسی استفاده از الگوریتم دیکسترا، نامنفی بودن وزن یال‌ها است. [۱]

۲-۴ پیچیدگی زمانی و حافظه‌ای

پیچیدگی زمانی الگوریتم دیکسترا به نوع ساختار داده مورد استفاده بستگی دارد. اگر از آرایه ساده برای انتخاب رأس با کمترین فاصله استفاده شود، پیچیدگی زمانی برابر است با:

$$T(n) = O(|V|^2) \quad (۳-۴)$$

اما اگر از صف اولویت و هیپ دودویی استفاده شود، پیچیدگی زمانی به صورت رابطه (۴-۴) خواهد بود:

$$T(n) = O((|V| + |E|) \log |V|) \quad (۴-۴)$$

همچنین پیچیدگی حافظه‌ای الگوریتم برابر است با:

$$S(n) = O(|V| + |E|) \quad (۵-۴)$$

طبق روابط (۳-۴) تا (۵-۴)، استفاده از ساختارهای داده مناسب می‌تواند عملکرد الگوریتم را بهبود دهد.

۳-۴ مزایا و محدودیت‌ها

الگوریتم دیکسترا دارای مزایای متعددی است که از جمله آن‌ها می‌توان به سادگی، کارایی مناسب و کاربرد گسترده در مسائل واقعی اشاره کرد. با این حال، این الگوریتم محدودیت‌هایی نیز دارد. مهم‌ترین

محدودیت آن، ناتوانی در کار با گراف‌هایی است که دارای یال با وزن منفی هستند.
به طور خلاصه:

• مزایا:

- سادگی درک و پیاده‌سازی،
- مناسب برای گراف‌های وزن‌دار نامنفی،
- کارایی خوب در کاربردهای عملی.

• محدودیت‌ها:

- عدم پشتیبانی از وزن‌های منفی،
- وابستگی کارایی به ساختار داده مورد استفاده.

فصل ۵

جمع‌بندی و نتیجه‌گیری و پیشنهادات

در این مقاله، الگوریتم دیکسترا به عنوان یک روش مهم برای حل مسئله کوتاه‌ترین مسیر در گراف‌های وزن‌دار نامنفی معرفی شد. ابتدا صورت مسئله و ایده اصلی الگوریتم بیان گردید، سپس شبه‌کد آن در الگوریتم ۱ ارائه شد. همچنین روابط ریاضی مربوط به طول مسیر، به‌روزرسانی فاصله و مقداردهی اولیه در روابط (۱-۳)، (۲-۳) بیان شدند. در ادامه نیز فلوچارت الگوریتم در شکل ۱-۳ ترسیم شد و پیچیدگی زمانی و حافظه‌ای آن بررسی گردید.

۱-۵ پیشنهادات

فرض نامنفی بودن وزن یال‌ها فرض مهم و حیاتی این الگوریتم است. اگر در مدلسازی مسئله نتوان تضمین کرد که تمامی یال‌ها وزنی مثبت دارند نباید از این الگوریتم استفاده کرد و باید سراغ الگوریتم‌های رفت که این فرض را ندارند مانند الگوریتم بلمن فورد.

منابع و مراجع

- [1] Proof of correctness. <https://web.engr.oregonstate.edu/~glencora/wiki/uploads/dijkstra-proof.pdf>. Accessed: 2015-12-03.